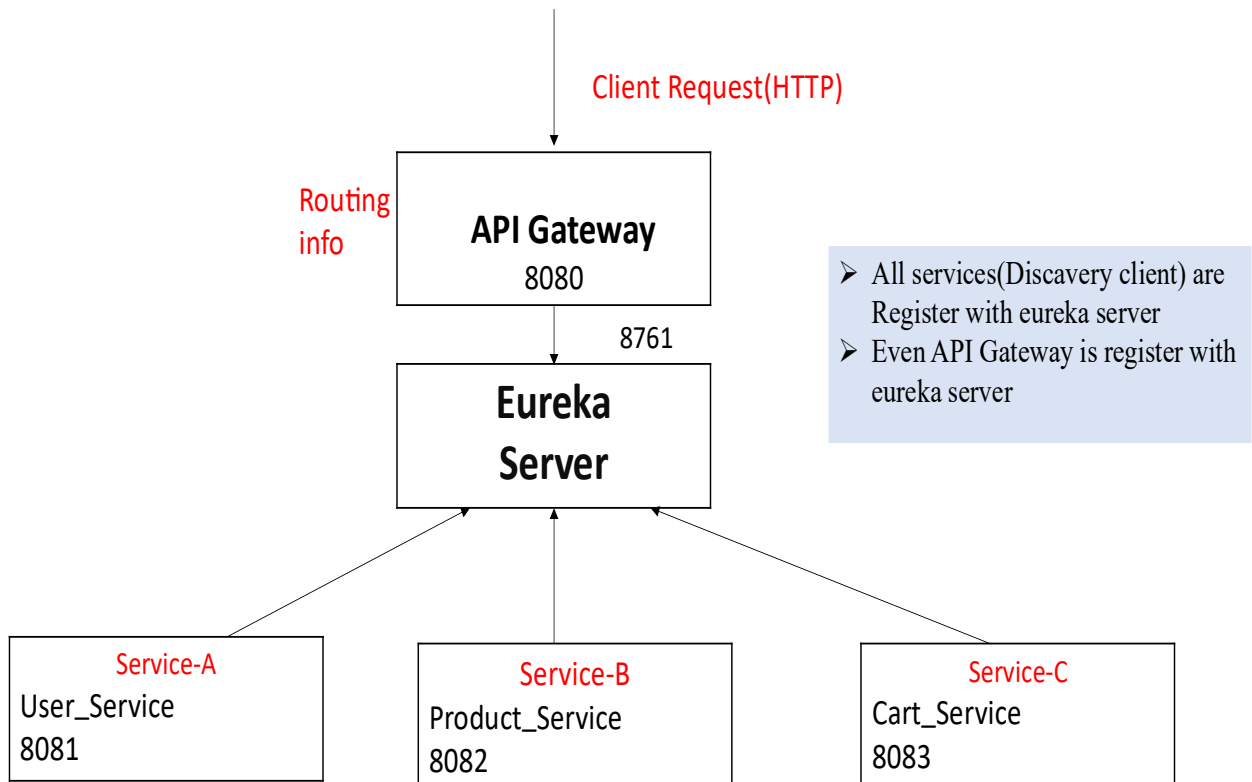


MICROSERVICES

→Microservices Architecture

It's an architecture style where a big application is divided into Small Independent Service.

- Each Services has its own logic and Database
- Runs Independently
- Communicate with each other via REST API's



Note: If one service crashes or fails, the other services will not be affected.

Example:

Think of **Flipkart**.

It has different services like:

- Product service
- Cart service
- Payment service

If the **Payment service stops working**, you can't pay.
But you can still:

- Search products

- Add to cart
- View your cart

Only the payment part is down.
The rest of the app works normally.

Because every service is separate in microservices.

But in Monolithic Application

Flipkart is one big code + one big database.
Product, Cart, Payment → everything is inside one **single big application**.

If the **payment part** has a problem, the **whole application can slow down or stop** because everything is tightly connected.

So, if Payment crashes:

- Product search may stop
- Cart may not work
- Whole app can go down
- Users may see “Server Down” for entire website

Important Components in Microservices

- Eureka Server
- Discovery Client
- API Gateway
- Config Server
- Admin Server

1) Eureka Server

If we have 10+ microservices (User, Order, Payment, etc.).
Each run's on a **different port**.

How will one service know where the other is running?

Eureka Server like a directory for microservices.

- Every service (like User-Service or Order-Service) **registers itself** to the Eureka Server.
- Eureka keeps track of all active services and their locations (hostname + port).
- When one service needs another, it asks Eureka for the address.

So even if User-Service changes its port or machine, Order-Service can still reach it using the **service name**, not the actual URL.

How to create eureka server?

Step 1: Create a Spring Boot Project using: Spring Initializr Or STS

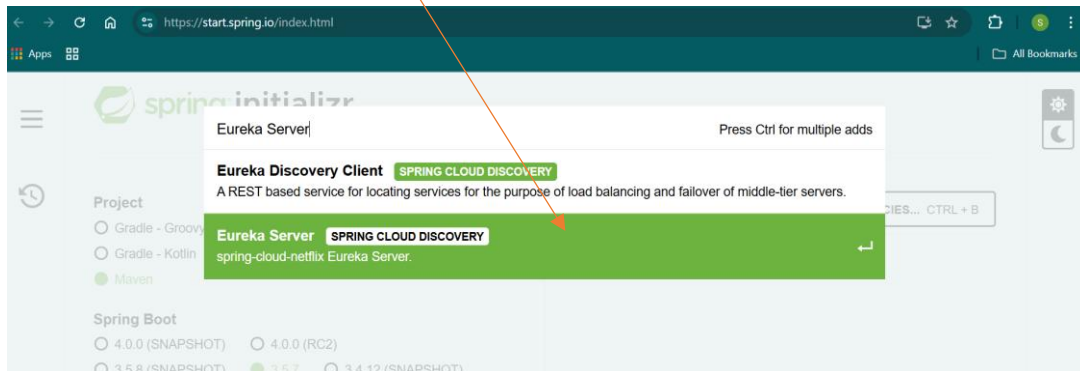
→ Add Eureka Server dependency

```
<dependency>
```

```
<groupId>org.springframework.cloud</groupId>
```

```
<artifactId>spring-cloud-starter-netflix-eureka-server</artifactId>
```

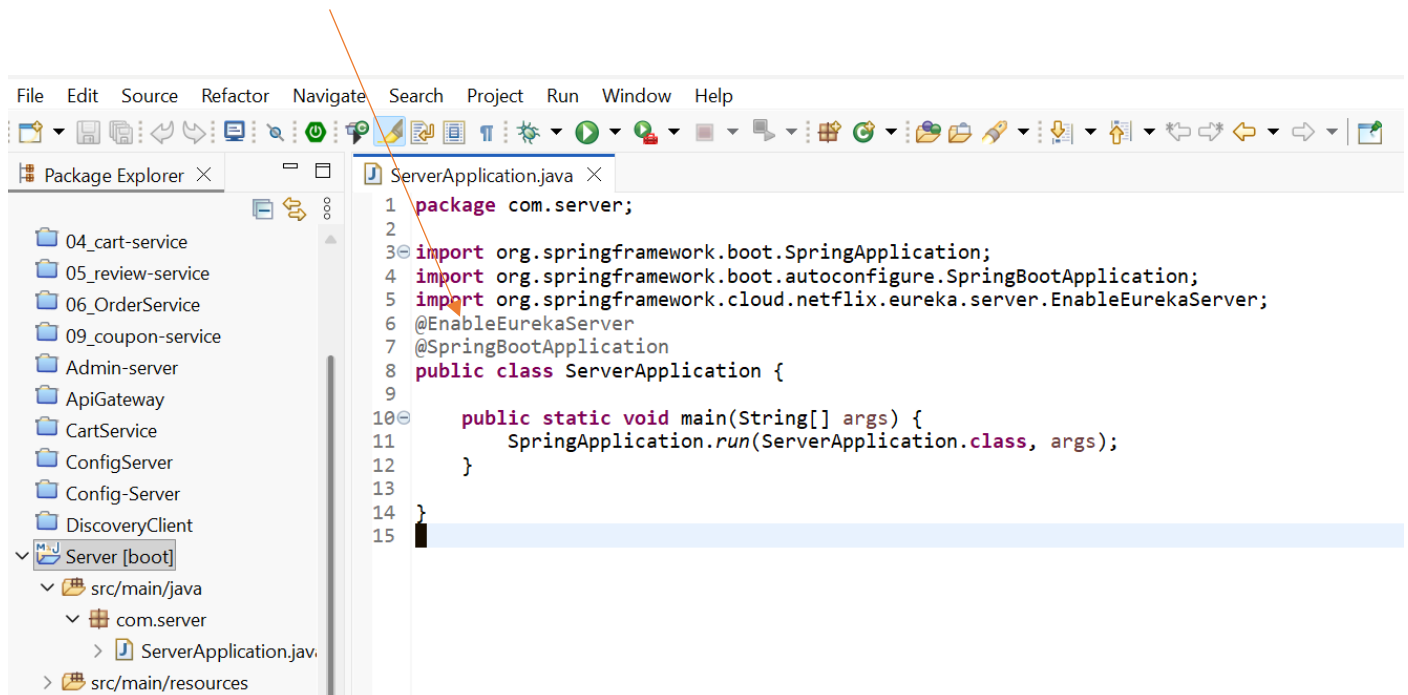
```
</dependency>
```



Note: If you created project in spring Initializr you need to import the project to STS.

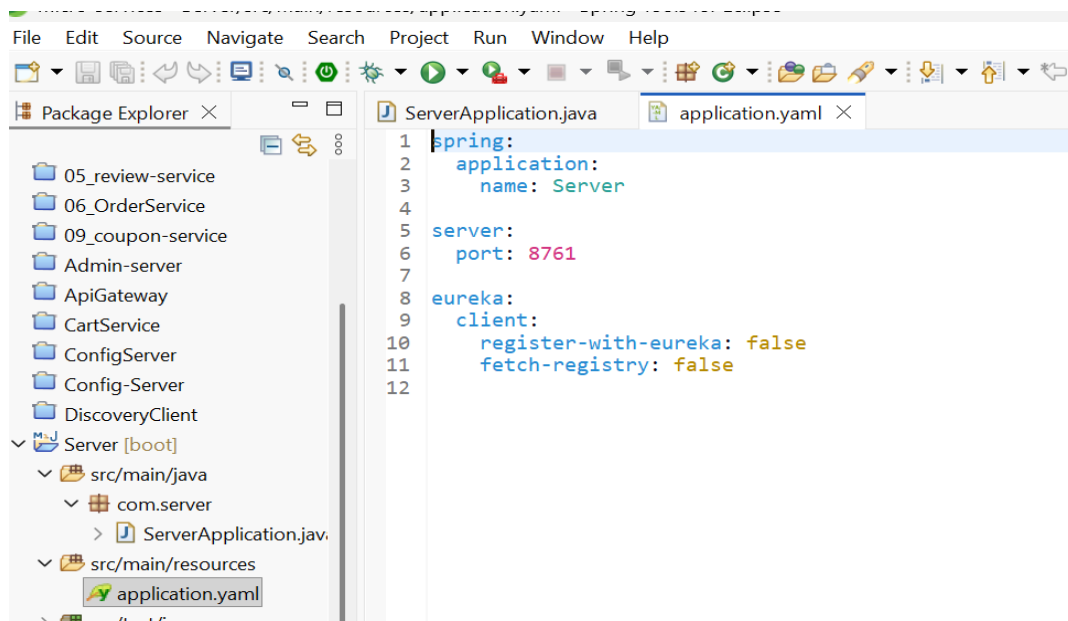
Step 2: Enable Eureka Server in Main Class

In your main class, add **@EnableEurekaServer**:



Step 3: In application.yaml (src/main/resources) make eureka client register with eureka false.

Eureka Server should NOT register with itself, so make eureka.client.register.eureka: false



Note:

If you create your project using Spring Initializr you will automatically get application.yaml inside src/main/resources

But if you create your project using STS (Spring Tool Suite), it gives you application.properties by default.

So in STS, you must manually create: application.yaml inside src/main/resources

2)Discovery Client.

Each services becomes a discovery client when it register to the eureka server

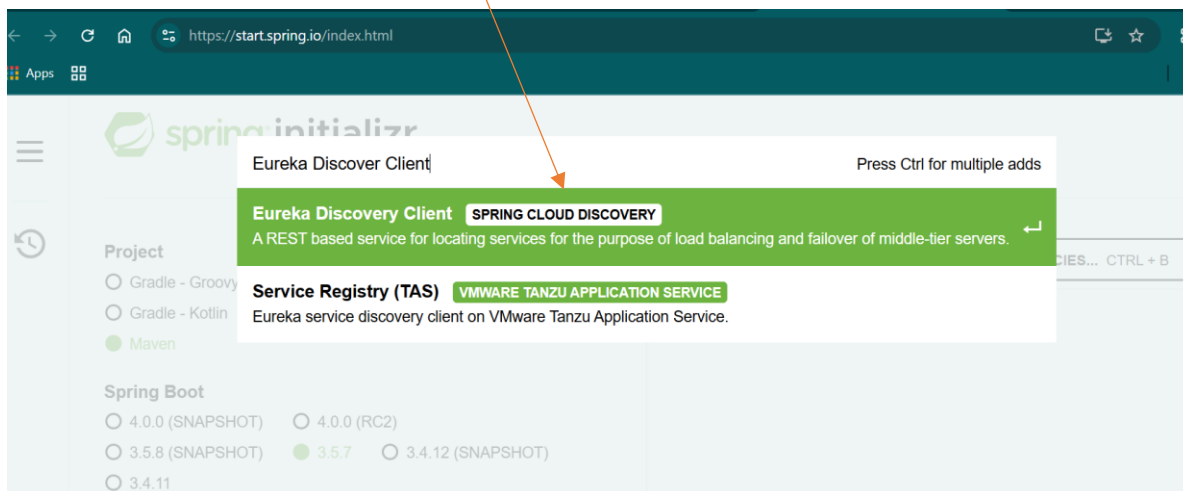
How Services Register to Eureka Server?

Step 1:

Create any microservice (like cart-service, user-service, payment-service) using: Spring Initializr Or STS

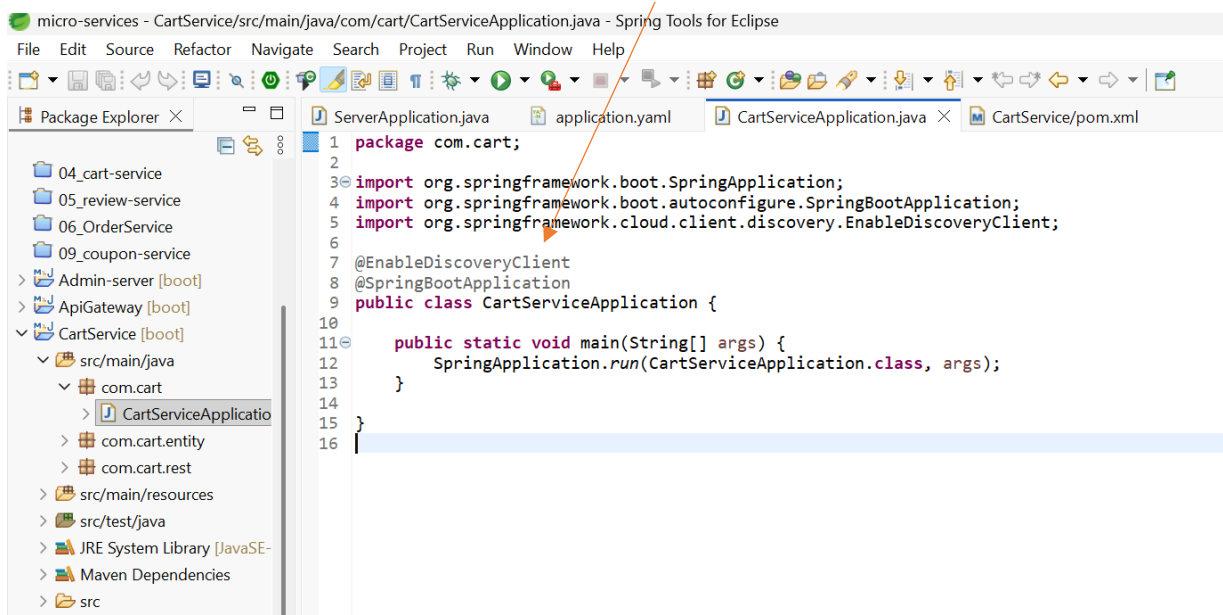
→Add **Eureka Discovery Client** dependency

```
<dependency>
<groupId>org.springframework.cloud</groupId>
<artifactId>spring-cloud-starter-netflix-eureka-client</artifactId>
</dependency>
```

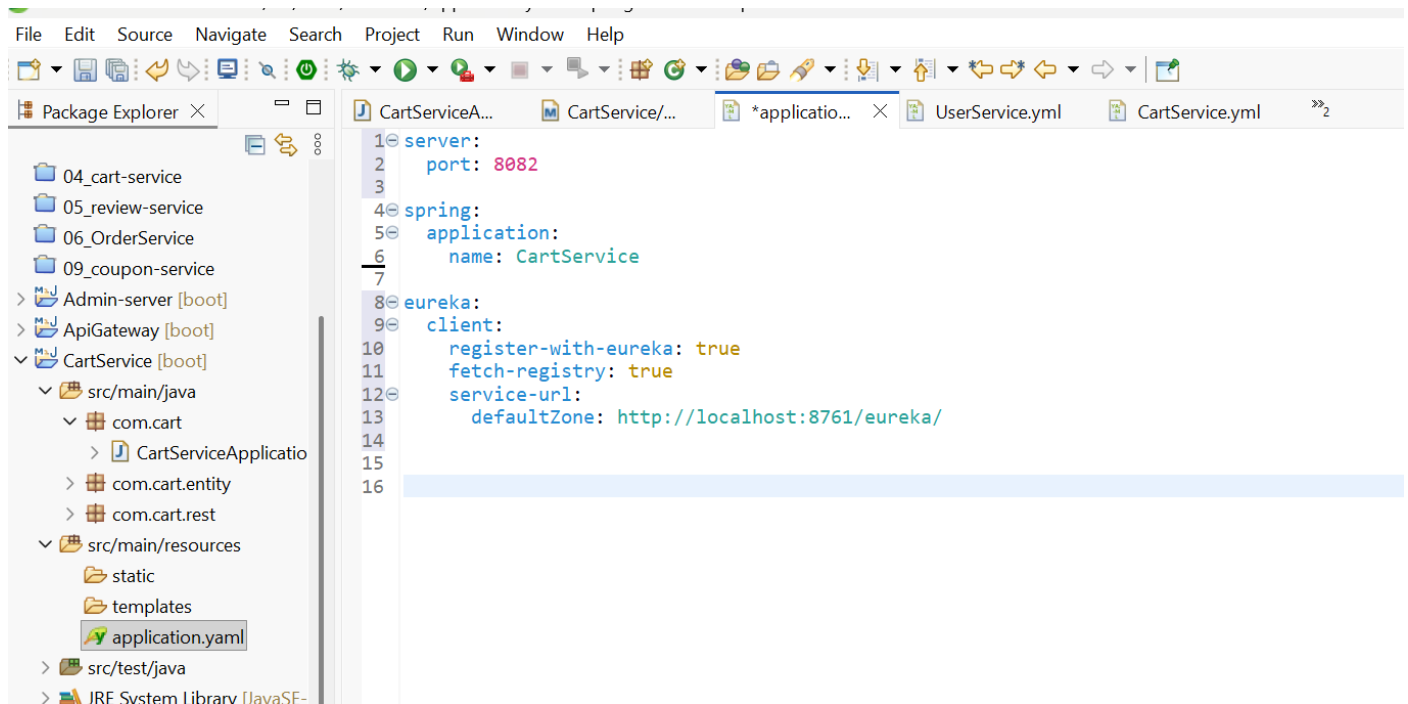


Step 2: Enable DiscoveryClient in Main Class

In your main class, add **@EnableDiscoveryClient**:



Step 3: Register the service(Discovery client/Eureka client) with eureka server and give eureka server URL:

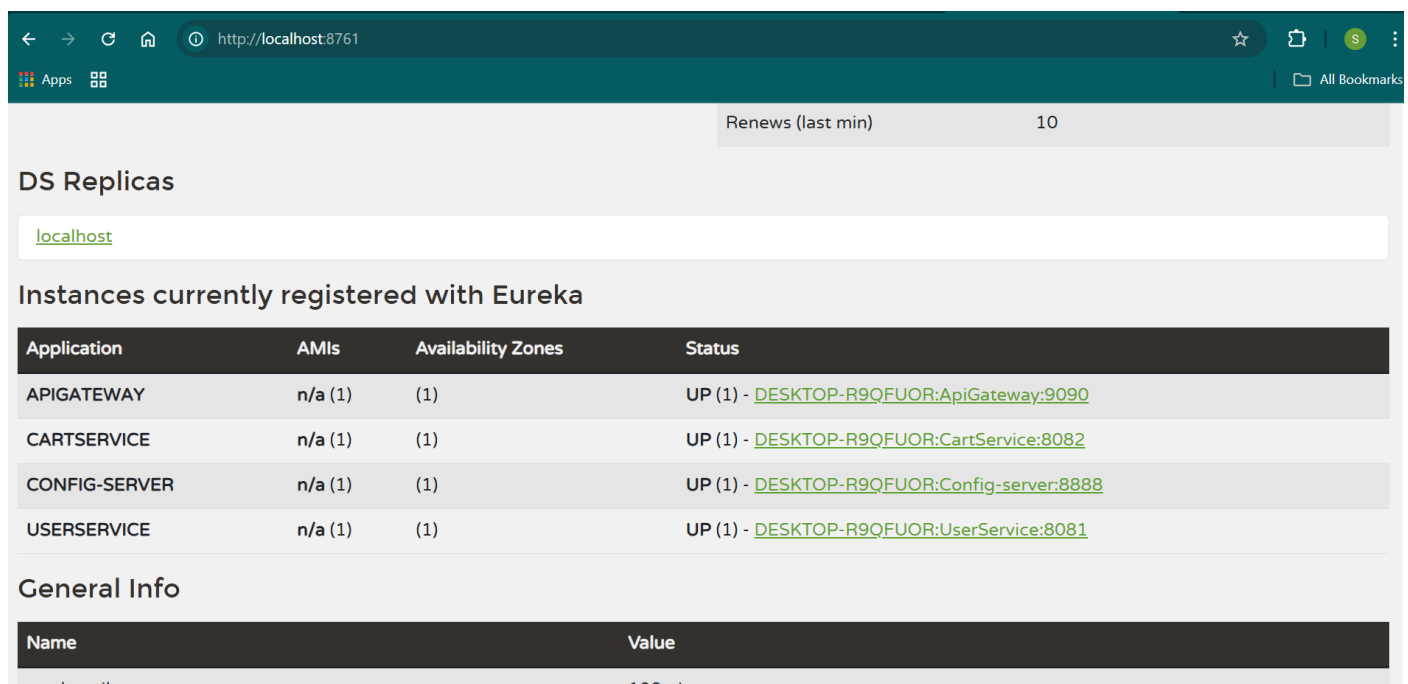


Now first run server then the services (like cart-service, payment-service etc).

As each service starts, it will automatically register to Eureka.

You can see them in the Eureka Dashboard:

<http://localhost:8761>



3) API Gateway

a single entry point (reverse proxy) for all client requests. It routes requests to internal services.

- Routes requests to the right service (based on Eureka)
- Handles authentication, logging, rate-limiting, and security
- Hides internal service URLs from clients (only Gateway URL is public)

How to create API Gateway?

Step 1: Create spring-boot project using: Spring Initializr Or STS

➔ Add **Eureka Discovery Client and Gateway** dependencies.

```
<dependency>
<groupId>org.springframework.cloud</groupId>
<artifactId>spring-cloud-starter-gateway-server-webmvc</artifactId>
</dependency>
<dependency>
<groupId>org.springframework.cloud</groupId>
<artifactId>spring-cloud-starter-netflix-eureka-client</artifactId>
</dependency>
```

Dependencies

ADD DEPENDENCIES... CTRL + B

Gateway **SPRING CLOUD ROUTING**

Provides a simple, yet effective way to route to APIs in Servlet-based applications. Provides cross-cutting concerns to those APIs such as security, monitoring/metrics, and resiliency.



Eureka Discovery Client **SPRING CLOUD DISCOVERY**

A REST based service for locating services for the purpose of load balancing and failover of middle-tier servers.



Step 2: Enable **DiscoveryClient** in Main Class (Even API Gateway need to register with eureka server)

In your main class, add **@EnableDiscoveryClient**:

Step 3:

→ Register to eureka server by making eureka client register with eureka true and give eureka server URL.

→ Enables **Spring Cloud Gateway** using **WebMVC mode** and defines routes.

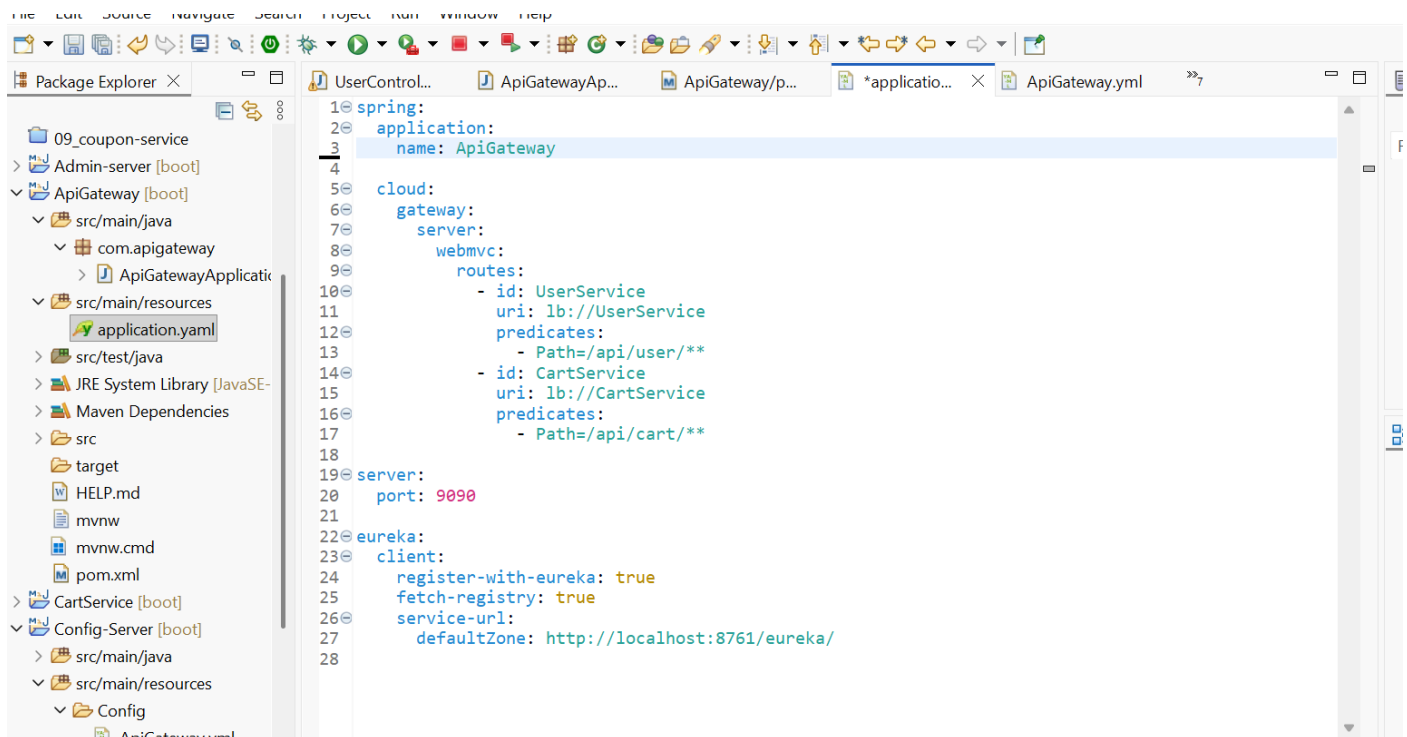
Ex: - `id: UserService`

`uri: lb://UserService`

`predicates:`

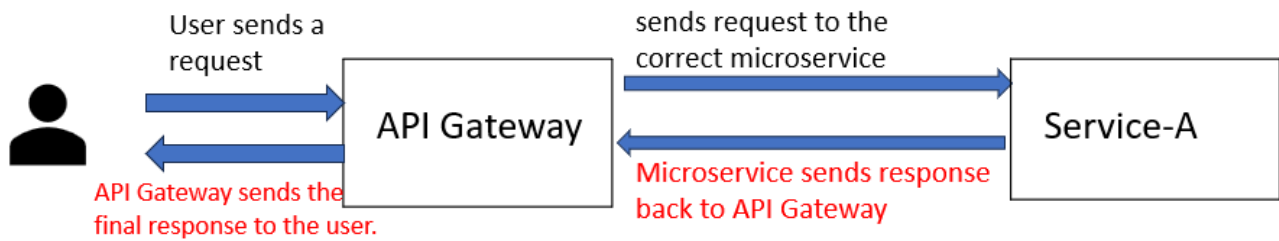
- `Path=/api/user/**`

- `id` → name of the route
- `uri: lb://UserService` → `lb://` means load balancing using Eureka. Gateway will forward the request to the service registered as `UserService` in Eureka.
- predicate `Path=/api/user/` → Any URL starting with `/api/user/` will be sent to `UserService`.



1. First start **Eureka Server**
2. Then start all **microservices** (`UserService`, `CartService`, `ProductService...`)
3. Finally start the **API Gateway**

How User Request Flows in Microservices?



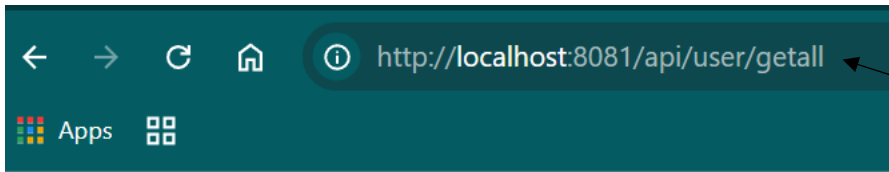
1. User sends a request
Example: "Show me all users." → `http://localhost:8082/api/user/getAll`
2. Request goes to API Gateway
API Gateway decides which service should handle the request.
3. API Gateway sends request to the correct microservice
Example: User request → User Service
Cart request → Cart Service
Payment request → Payment Service
4. That microservice processes the request
Using its own logic and its own database.
5. Microservice sends response back to API Gateway.
6. API Gateway sends the final response to the user.

```
UserController.java
1 package com.user.rest;
2
3 import org.springframework.beans.factory.annotation.Autowired;
4
5 @RestController
6 @RequestMapping("/api/user")
7 public class UserController {
8
9     //consider we have a method which gives alluser
10    @GetMapping("/getall")
11    public String getAllUser() {
12        return "All User Data fetched";
13    }
14
15 }
```

Note:

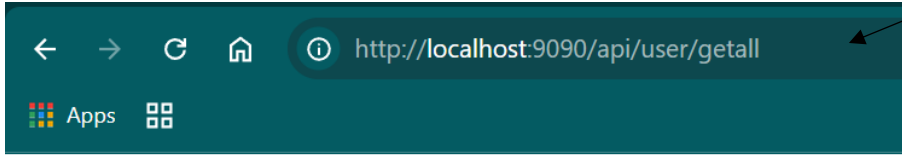
`http://localhost:8082/api/user/getAll` (using service port number) OR

`http://localhost:9090/api/user/getAll` (using API Gateway port number) → goes to UserService.



All User Data featched

Getting the same result
from API Gateway
(9090) and the service
(8081)



All User Data featched

4) Config Server (Centralized Configuration)

Spring Cloud Config Server used to stores all configurations (YAML files) in one central place.

1) We can store all YAML files inside the Config Server by creating a folder in src/main/resources.

2) Or we can create a Git repository and store all the YAML files in that repo.

→ All microservices fetch their config from the Config Server when application starts.

→ If you update the config in Git, you can refresh it for all services without restarting them.

With Config Server

You do NOT change YAML inside each microservice.

Instead, you keep all YAML files in **one Git repo (or) one folder in config-server**, like:

config-repo/

├─ user-service.yml

├─ order-service.yml

├─ payment-service.yml

└─ cart-service.yml

Now if you want to change anything:

- You change YAML only in Git repo
- You do not open microservice projects
- You do not restart microservices
- You just refresh config

Without Config Server

Each microservice has its own application.yml.

If you want to change something:

- Stop the Service
- Open particular Service
- Edit YAML files
- Restart microservices

Imagine if we want to change 5 services we have to do the above steps 5 times.

How to create Config-server?

Step1: Create spring-boot project using: Spring Initializr Or STS

→ Add **Eureka Discovery Client and Config Server** dependencies

```
<dependency>
<groupId>org.springframework.cloud</groupId>
<artifactId>spring-cloud-config-server</artifactId>
</dependency>
<dependency>
<groupId>org.springframework.cloud</groupId>
<artifactId>spring-cloud-starter-netflix-eureka-client</artifactId>
</dependency>
```

Dependencies

ADD DEPENDENCIES... CTRL + B

Config Server

SPRING CLOUD CONFIG

Central management for configuration via Git, SVN, or HashiCorp Vault.

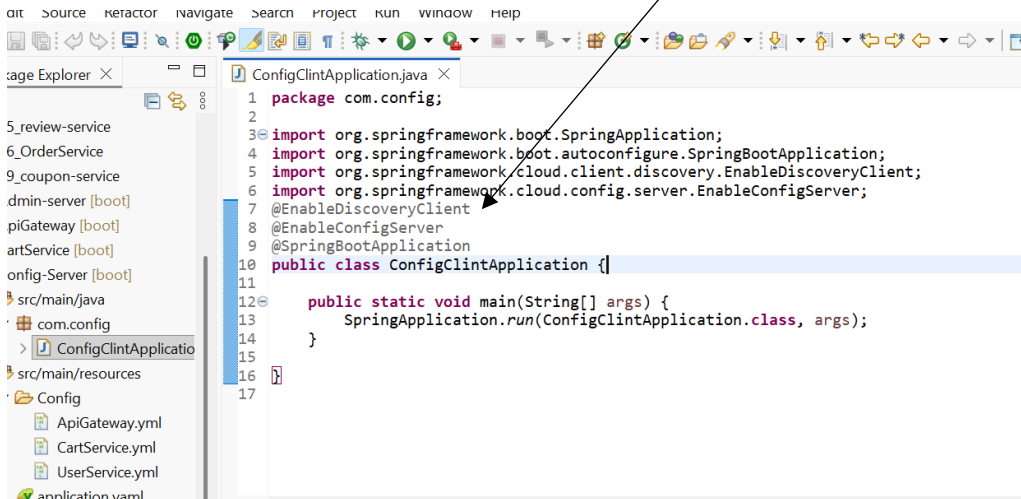
Eureka Discovery Client

SPRING CLOUD DISCOVERY

A REST based service for locating services for the purpose of load balancing and failover of middle-tier servers.

Step2: Enable DiscoveryClient and ConfigServer in Main Class.

In your main class, add `@EnableDiscoveryClient` ,`@EnableConfigServer`:



```
1 package com.config;
2
3 import org.springframework.boot.SpringApplication;
4 import org.springframework.boot.autoconfigure.SpringBootApplication;
5 import org.springframework.cloud.client.discovery.EnableDiscoveryClient;
6 import org.springframework.cloud.config.server.EnableConfigServer;
7 @EnableDiscoveryClient
8 @EnableConfigServer
9 @SpringBootApplication
10 public class ConfigClientApplication {
11
12     public static void main(String[] args) {
13         SpringApplication.run(ConfigClientApplication.class, args);
14     }
15 }
16
17
```

Step3: Configure application.yml

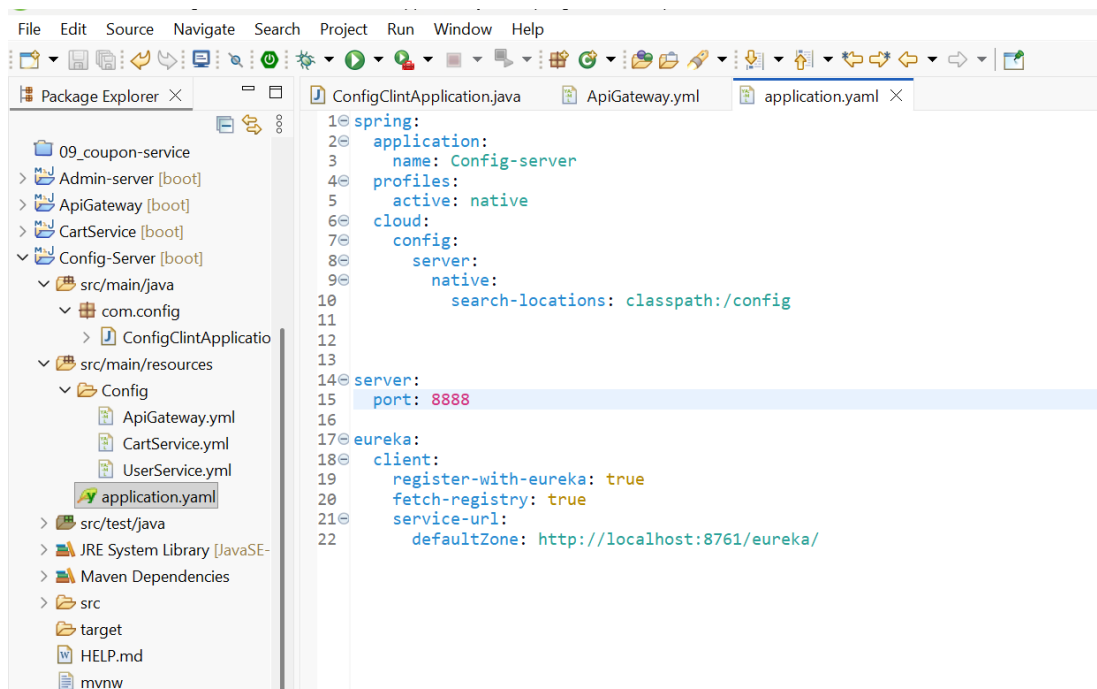
In the Config Server's application.yml.

→Register the Config Server with **Eureka**

→Specify where the **central config folder** (that contains all microservice YAML files) is located

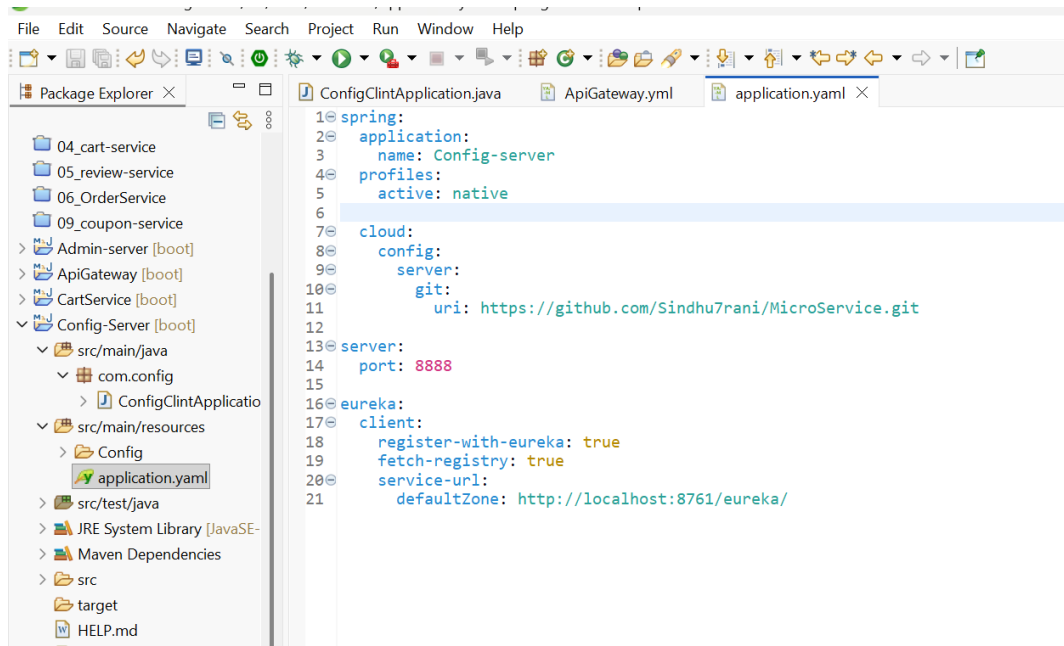
(either in src/main/resources or in a **Git repository**)

If you stored all yaml in src/main/resources→give folder address.



```
1 spring:
2   application:
3     name: Config-server
4   profiles:
5     active: native
6   cloud:
7     config:
8       server:
9         native:
10           search-locations: classpath:/config
11
12
13
14 server:
15   port: 8888
16
17 eureka:
18   client:
19     register-with-eureka: true
20     fetch-registry: true
21     service-url:
22       defaultZone: http://localhost:8761/eureka/
```

If you stored all yaml in git repo → give repo URL.



Step4: Add Config Client dependency in all microservices

Every microservice must include the **Spring Cloud Config Client** dependency so it can connect to the Config Server and fetch its YAML file.

```
<dependency>
<groupId>org.springframework.cloud</groupId>
<artifactId>spring-cloud-starter-config</artifactId>
</dependency>
```

Dependencies

ADD DEPENDENCIES... CTRL + B

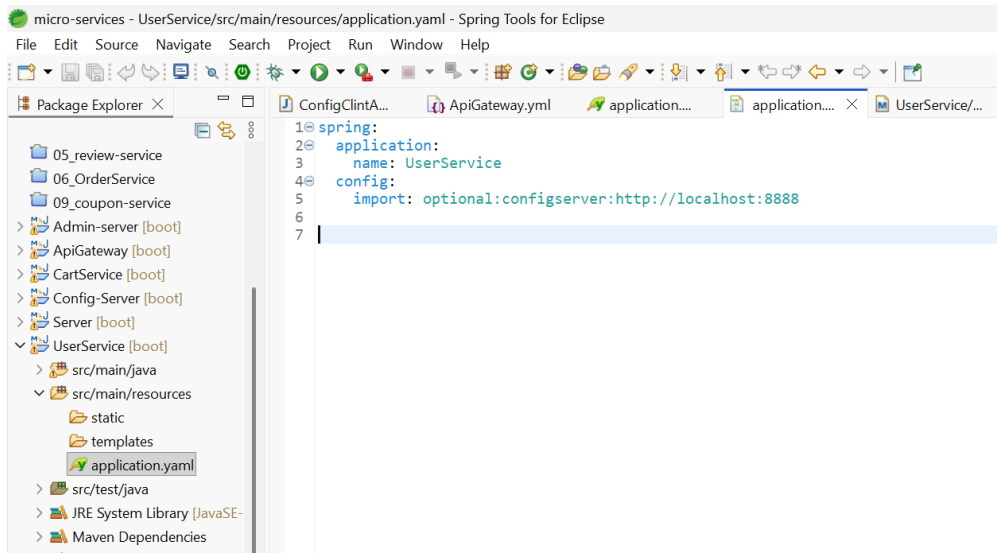
Config Client **SPRING CLOUD CONFIG**

Client that connects to a Spring Cloud Config Server to fetch the application's configuration.

Step 5: Add **spring.config.import:optional:configserver:http://localhost:8888** in the application.yaml file of all microservices.

Because, each microservice needs to know **where the Config Server is running** so that it can fetch its configuration (like port number, Eureka settings, DB settings) from the Config Server at startup.

So, inside **every microservice's application.yaml**, we must add:



5) Admin Server (Monitoring Dashboard)

Spring Boot Admin Server helps to monitor all microservices from one dashboard.

It shows Which services are up/down, Memory, CPU, uptime, Logs and environment details

How to create Admin Server and how can microservices register to Admin Server?

Step 1: Create spring-boot project using: Spring Initializr Or STS.

Add Spring Boot **Admin Server** and **Actuator** dependencies.

```
<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-actuator</artifactId>
</dependency>
<dependency>
<groupId>de.codecentric</groupId>
<artifactId>spring-boot-admin-starter-server</artifactId>
</dependency>
```

Actuator gives endpoints like:

- /actuator/health
- /actuator/info
- /actuator/metrics
- /actuator/loggers
- /actuator/env

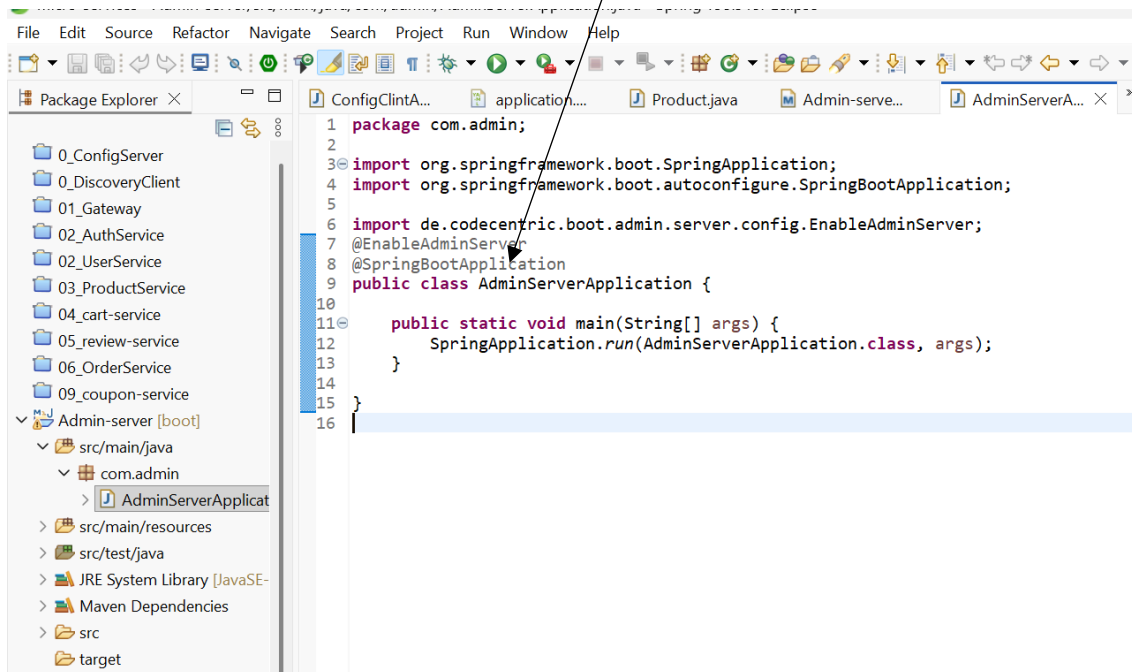
These endpoints help **Admin Server** to monitor your microservices.

Spring boot Actuator → gives live info about each microservices

Spring boot admin server → Displays that live info

Step2: Enable Admin Server in Main Class.

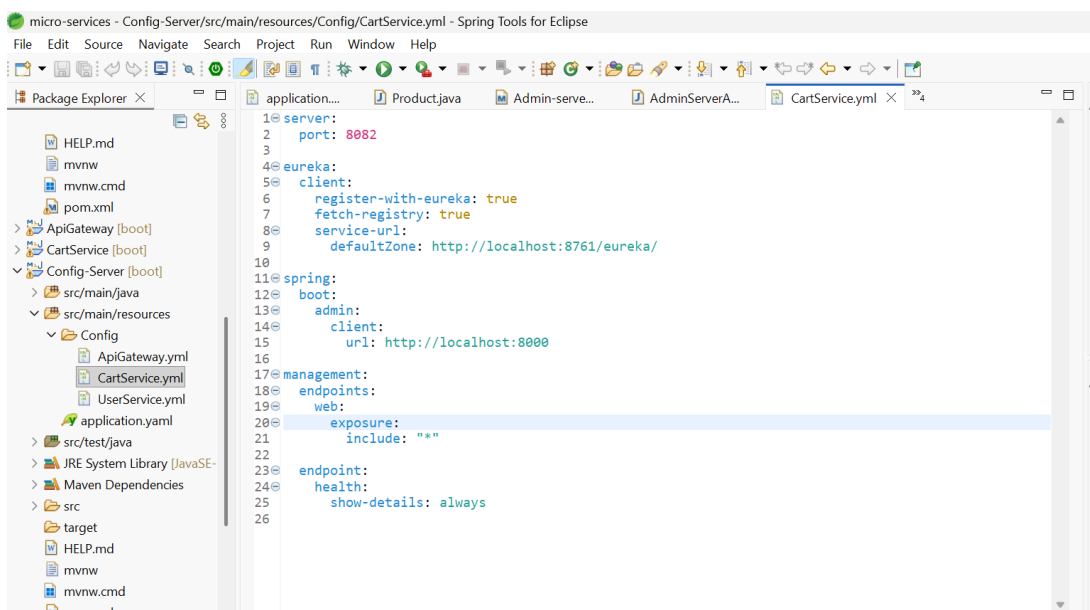
In your main class, add **@EnableAdminServer**



```
1 package com.admin;
2
3 import org.springframework.boot.SpringApplication;
4 import org.springframework.boot.autoconfigure.SpringBootApplication;
5
6 import de.codecentric.boot.admin.server.config.EnableAdminServer;
7 @EnableAdminServer
8 @SpringBootApplication
9 public class AdminServerApplication {
10
11     public static void main(String[] args) {
12         SpringApplication.run(AdminServerApplication.class, args);
13     }
14 }
15
16
```

Step3: In each microservice's application.yml, add spring:boot:admin:client:url: http://localhost:8000

So, Every microservice sends its health, metrics, info, logs, etc. to the Admin Server, so Admin can monitor them.



```
1 server:
2   port: 8082
3
4 eureka:
5   client:
6     register-with-eureka: true
7     fetch-registry: true
8     service-url:
9       defaultZone: http://localhost:8761/eureka/
10
11 spring:
12   boot:
13     admin:
14       client:
15         url: http://localhost:8000
16
17 management:
18   endpoints:
19     web:
20       exposure:
21         include: "*"
22
23 endpoint:
24   health:
25     show-details: always
26
```

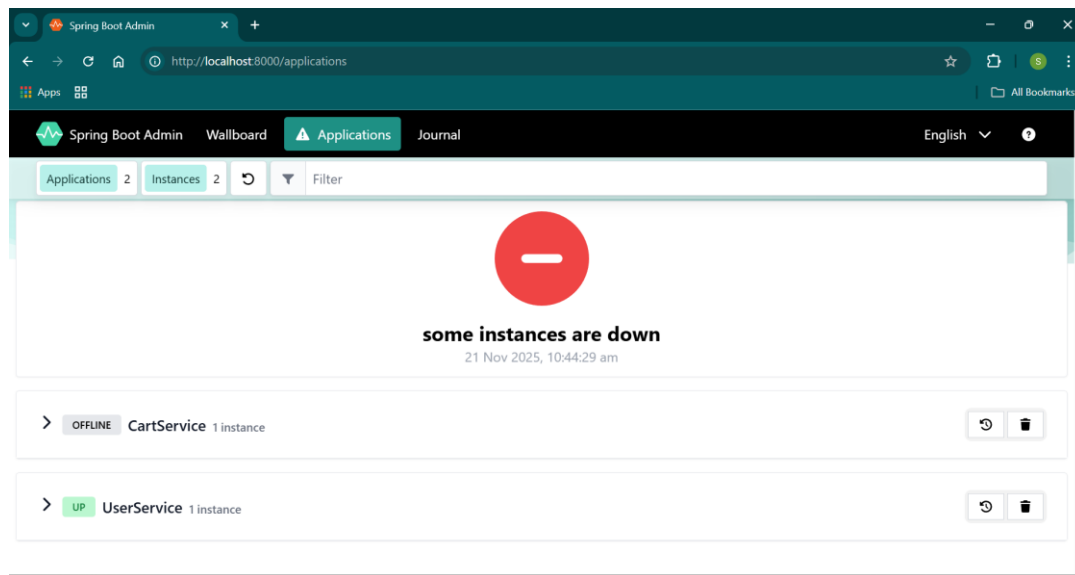
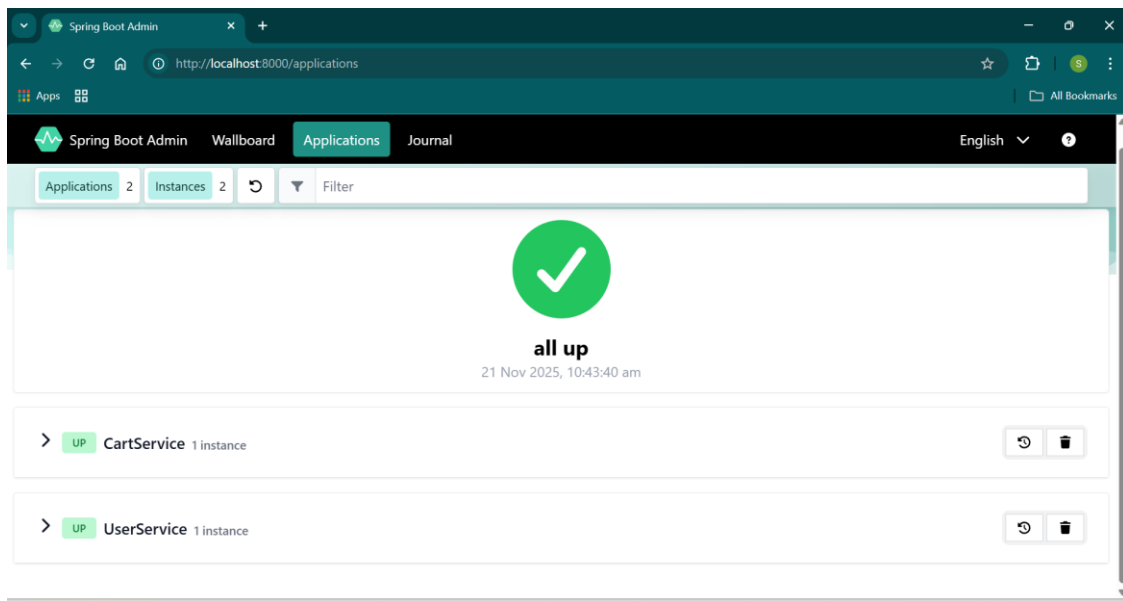
→ **management:endpoints:web:exposure:include: "*"**

By default, actuator hides many endpoints.

This line **opens all endpoints** so Admin Server can monitor the service.

→ **management:endpoint:health:show-details: always**

Show full health info (required by Admin Server)



Order Of Execution.

- 1) Eureka Server:** All microservices and API Gateway must register here.
- 2) Config Server:** Config Server provides application.yaml to all microservices.
- 3) Admin Server:** Admin Server monitors services after they register to Eureka.
- 4) API Gateway:** It uses Eureka to find services, so it should start after Eureka.
- 5) All Microservices:** They depend on all the above, so they start last.

How to communicate between two services?

Microservices communicate with each other using REST APIs.

In Spring Boot, we use **Feign Client** to easily call the API of one service from another service.

- Feign acts as a declarative HTTP client.
- Feign makes communication easy because we write only an interface, not full REST code and call remote services like normal methods.

Example:

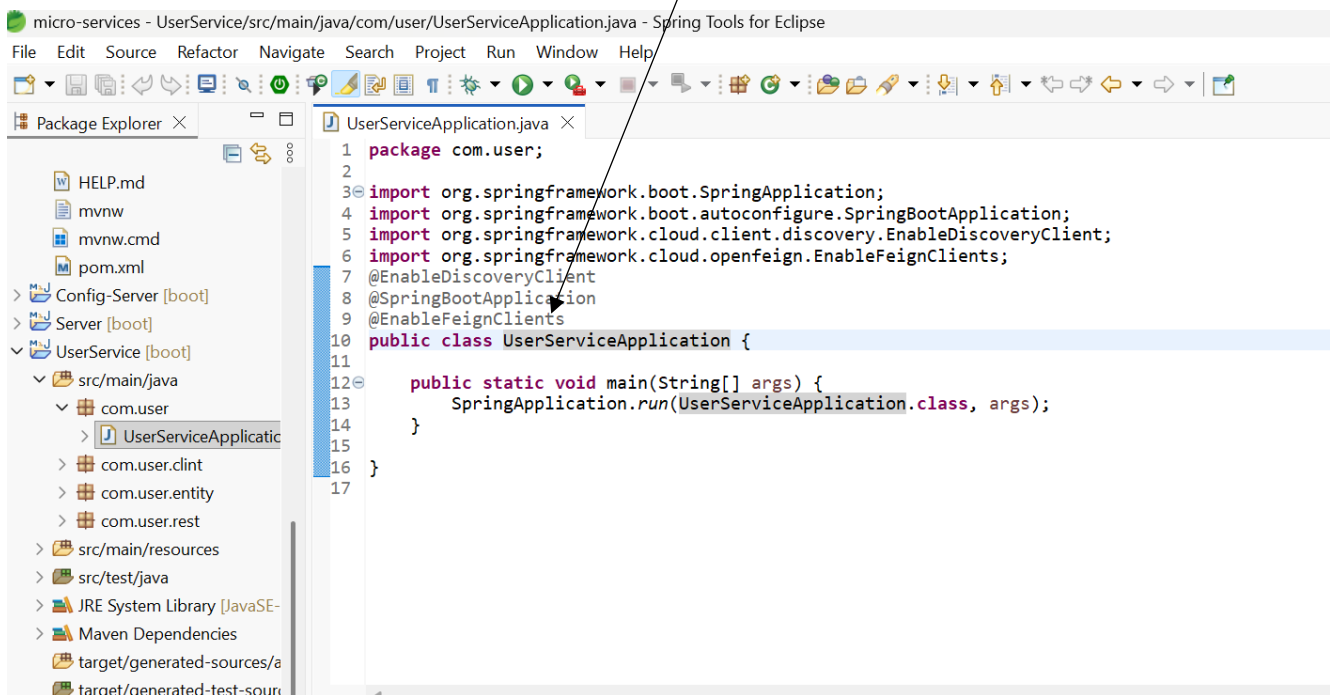
User Service wants product details from Product Service.

Step1: Add **openfeign** dependency in microservice which want to communicate (in our example User Service)

```
<dependency>
<groupId>org.springframework.cloud</groupId>
<artifactId>spring-cloud-starter-openfeign</artifactId>
</dependency>
```

Step2: Enable feignclient in Service which wants to communicate

In UserService main class Add **@EnableFeignClients**



Step 4: Create the Feign Client inside the service that wants to communicate with another microservice.

Here, in our example, the User Service needs to communicate with the Product Service, so we have to create the Feign Client interface inside the User Service.

Add **@FeignClient(name="K-ProductService")**

- name = "K-ProductService" → is the service name registered in Eureka
- Write the same name that your ProductService uses in its spring.application.name

This tells User Service which service it should talk to.

The screenshot shows the Eclipse IDE with the 'ProductFeignClient.java' file open in the 'com.user.feign' package. The code defines the interface with the following content:

```
1 package com.user.feign;
2
3 import java.util.List;
4
5 import org.springframework.cloud.openfeign.FeignClient;
6 import org.springframework.web.bind.annotation.GetMapping;
7
8 import com.user.dto.ProductDto;
9
10
11 @FeignClient(name = "K-ProductService")
12 public interface ProductFeignClient {
13
14
15
16
17     @GetMapping("/api/product/all")
18     public List<ProductDto> getAllProducts();
19
20
21 }
22
```

The Package Explorer on the left shows the project structure, with 'ProductFeignClient.java' selected under 'com.user.feign'.

Inside your Feign client, create a method whose:

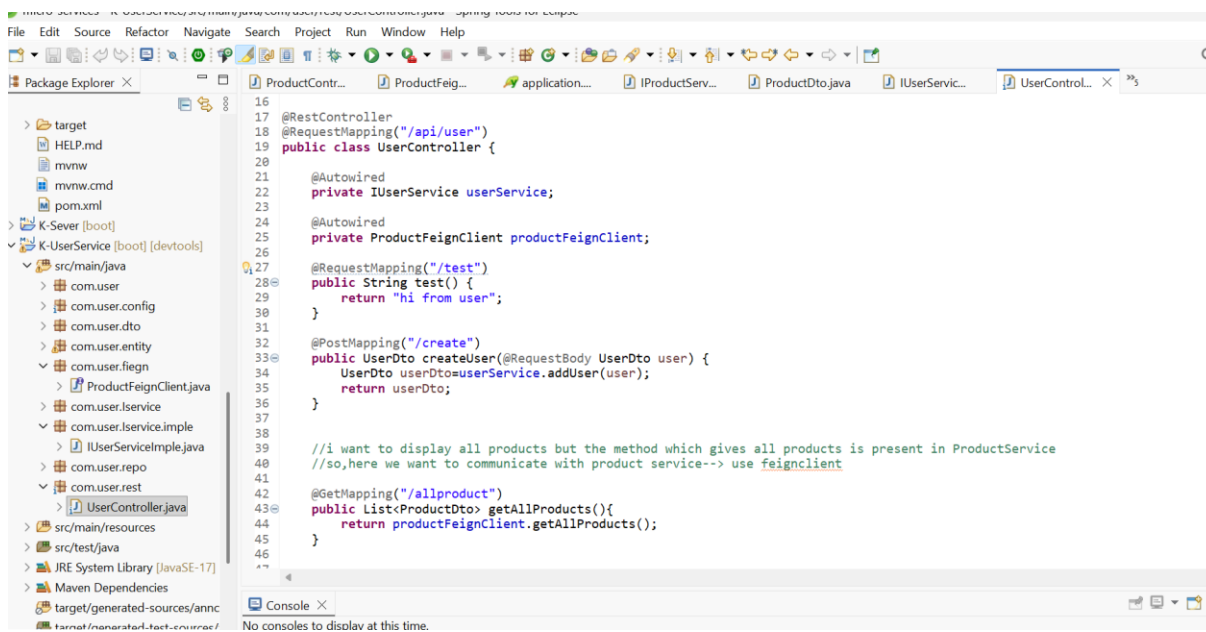
- URL path must match the API endpoint in ProductService
- HTTP method must match (@GetMapping, @PostMapping, etc.)
- Input parameters must match (like @PathVariable, @RequestParam)
- Return type must match the response of ProductService

The screenshot shows the Eclipse IDE with the 'ProductController.java' file open in the 'com.product.rest' package. The code implements the controller with the following content:

```
7 import org.springframework.web.bind.annotation.PostMapping;
8 import org.springframework.web.bind.annotation.RequestBody;
9 import org.springframework.web.bind.annotation.RequestMapping;
10 import org.springframework.web.bind.annotation.RestController;
11
12 import com.product.IService.IProductService;
13 import com.product.dto.ProductDto;
14
15
16 @RestController
17 @RequestMapping("/api/product")
18 public class ProductController {
19
20     @Autowired
21     private IProductService productService;
22
23     @PostMapping("/create")
24     public ProductDto createProduct(@RequestBody ProductDto productDto) {
25         ProductDto res=productService.addProduct(productDto);
26         return res;
27     }
28
29
30     @GetMapping("/all")
31     public List<ProductDto> getAllProducts(){
32         return productService.getAllProduct();
33     }
34
35 }
36
```

The Package Explorer on the left shows the project structure, with 'ProductController.java' selected under 'com.product.rest'.

If you want to communicate with another service, simply autowire the Feign Client in your service and call its method, and Spring Cloud OpenFeign will internally map this call to the corresponding method in the other microservice and execute it.



```
16
17 @RestController
18 @RequestMapping("/api/user")
19 public class UserController {
20
21     @Autowired
22     private IUserService userService;
23
24     @Autowired
25     private ProductFeignClient productFeignClient;
26
27     @RequestMapping("/test")
28     public String test() {
29         return "hi from user";
30     }
31
32     @PostMapping("/create")
33     public UserDto createUser(@RequestBody UserDto user) {
34         UserDto userDto=userService.addUser(user);
35         return userDto;
36     }
37
38     //i want to display all products but the method which gives all products is present in ProductService
39     //so,here we want to communicate with product service--> use feignclient
40
41     @GetMapping("/allproduct")
42     public List<ProductDto> getAllProducts(){
43         return productFeignClient.getAllProducts();
44     }
45
46
47 }
```

A request is sent from the UserController to get all product details from the ProductService, and this confirms that inter-service communication is successfully activated.

Output: (<http://localhost:9090/api/user/allproducts>)

